

VISION BOARD WEB APP

Technical Architecture & Stack Decision Record

AWS-first, serverless-first architecture for a fast, media-heavy, extensible vision-board product

Core architecture principle

Keep application metadata relational, keep media in object storage, keep the canvas document compact, move slow work to asynchronous jobs, and never make the first meaningful render depend on downloading the entire world.

Area	Primary decision
Frontend	Next.js + React + TypeScript
Canvas	tldraw SDK
Database	Amazon Aurora PostgreSQL Serverless v2
Media	Amazon S3 + CloudFront
API/compute	AWS Lambda + API Gateway
Auth	Amazon Cognito
Async processing	SQS + Lambda
Observability	CloudWatch + Sentry
Deployment	Vercel for frontend; AWS for backend/data
EC2	Not required for MVP or normal production operation

Contents

- 1. Executive decision summary
- 2. Product requirements that drive architecture
- 3. Final stack and service decision matrix
- 4. Target system architecture
- 5. Frontend and canvas architecture
- 6. Data model and persistence strategy
- 7. Media upload, delivery, and image processing
- 8. API and compute architecture
- 9. Performance design and the 750-post lesson
- 10. Authentication, authorization, and security
- 11. Async jobs and event-driven workflows
- 12. Realtime and collaboration strategy
- 13. AI-ready architecture
- 14. Observability, reliability, and SLOs
- 15. Cost and scaling strategy
- 16. MVP-to-production roadmap
- 17. Anti-patterns and decisions to avoid
- 18. Architecture decision records (ADRs)
- 19. Launch checklist
- Appendix A. Suggested schema
- Appendix B. Suggested API surface
- Appendix C. Performance investigation checklist

1. Executive decision summary

Recommended baseline

Build the product as a TypeScript web application using Next.js and tldraw, backed by AWS managed services. Use Aurora PostgreSQL for structured product data and S3/CloudFront for board media. Use Lambda for stateless APIs and SQS for slow/background work. Avoid EC2 unless a future workload truly requires a continuously running server.

The architecture is intentionally optimized for the product characteristics that matter most: a highly interactive canvas, many user-uploaded images, fast board opening, autosave, templates, future collaboration, and optional AI generation. The core design avoids coupling those concerns together.

Decision area	Selected option	Reason	Decision status
Web framework	Next.js + React + TypeScript	Fast product development, strong React ecosystem, server/client flexibility, shared types.	Adopt
Canvas/editor	tldraw SDK	Avoid building zoom/pan/selection/transform/undo/redo from scratch.	Adopt
Primary database	Aurora PostgreSQL Serverless v2	Relational model fits users, boards, sharing, templates, goals, subscriptions and permissions.	Adopt
Media storage	S3	Designed for durable object storage; do not place uploaded images in DB rows.	Adopt
Media delivery	CloudFront	CDN delivery reduces latency and origin traffic.	Adopt
Backend compute	Lambda	No server management, scales per request, good fit for API and event work.	Adopt
API front door	API Gateway	Managed routing, throttling, auth integration and Lambda invocation.	Adopt
Authentication	Cognito	AWS-native managed user identity and token issuance.	Adopt
Async jobs	SQS + Lambda	Separates user-facing latency from image processing, exports, AI jobs and emails.	Adopt

Decision area	Selected option	Reason	Decision status
Realtime multiplayer	Defer from MVP	Adds architectural complexity; first validate single-user board value and persistence.	Defer
EC2	Do not use by default	Most workloads are stateless or event-driven and fit managed/serverless services.	Avoid by default

2. Product requirements that drive architecture

Requirement	Architectural implication
Interactive infinite canvas	Users drag, resize, rotate, group, layer, zoom and pan hundreds of elements. The editor must remain responsive independently of backend latency.
Media-heavy boards	Images dominate bytes transferred. The architecture must keep media out of API responses and databases.
Fast perceived load	A user should see the board shell and meaningful content quickly, while heavier images load progressively.
Autosave and recovery	Edits need durable persistence without blocking interaction or writing on every pointer movement.
Templates	Template cloning should create a board quickly without duplicating unnecessary media.
Sharing and permissions	The model should support owner/editor/viewer roles even if collaboration launches later.
Future AI workflows	Generation, image processing and summarization should run asynchronously and output normal board objects.
Serverless preference	The team generally avoids EC2; the stack should favor managed services and functions.
Scalability without premature complexity	Start with a small number of services, introduce specialized systems only when metrics justify them.

3. Final stack and service decision matrix

Layer	Decision	Alternatives considered	Why this decision	Status
Frontend	Next.js + React + TypeScript	Vite SPA + separate API; Remix	One codebase, SSR/SPA options, strong ecosystem, shared types.	Use
Design system	Tailwind CSS + shadcn/ui	Material UI; custom CSS	Fast iteration without locking the product into a heavy visual system.	Use
Canvas	tldraw SDK	Konva/Fabric.js; fully custom canvas	Most editor primitives already exist; supports custom shapes.	Use
Primary DB	Aurora PostgreSQL Serverless v2	DynamoDB; RDS fixed instance	Relational domain model and SQL flexibility; managed scaling.	Use
Object storage	S3	Database blobs; local disk	Durable, cheap, scalable object storage.	Use
CDN	CloudFront	Direct S3 URLs	Lower latency, caching, custom domain and access control.	Use
Backend	Lambda	EC2; long-running Node server	Stateless request/event workloads; no instance administration.	Use
API gateway	API Gateway	Direct Lambda URLs; custom reverse proxy	Central auth, routing, rate limits and observability.	Use
Authentication	Cognito	Custom auth; Auth0	AWS-managed identity and token verification.	Use
Async queue	SQS	Doing work inside request; database polling	Reliable decoupling and retries.	Use
Image transforms	Lambda initially	Always serve originals	Pre-generate thumbnails/optimized variants.	Use
Email	SES	SMTP server	Managed transactional email.	Later / use as needed
Analytics	PostHog	Build analytics internally	Fast event/funnel visibility; keeps product analytics separate from CloudWatch.	Use

Layer	Decision	Alternatives considered	Why this decision	Status
App errors	Sentry	CloudWatch only	Better frontend stack traces and release diagnostics.	Use
Infra as code	AWS CDK or Terraform	Manual console-only setup	Reproducible environments and reviewable changes.	Use
Realtime	Defer; then managed sync or Fargate if self-hosting	DIY WebSocket server on EC2	Do not pay complexity tax until collaboration is required.	Defer

4. Target system architecture

The architecture deliberately separates the interactive editor, structured application state, media, and background processing.

```

USER BROWSER
|
+--> Next.js UI / tldraw editor
|
|   +--> Cognito (sign-in / tokens)
|   |
|   +--> API Gateway --> Lambda --> Aurora PostgreSQL
|   |                                     |
|   |                                     +--> SQS
|   |                                     +--> worker Lambda
|   |                                     +--> transforms / exports
|   |                                     +--> AI / email
|   |
|   +--> request presigned upload URL
|   |   |
|   |   +--> direct upload to S3
|   |
|   +--> CloudFront --> S3 optimized images / thumbnails / exports

```

Critical rule

Large media should travel directly between the browser and S3/CloudFront. Lambda should authorize and coordinate media operations, not proxy every image byte.

5. Frontend and canvas architecture

Concern	Decision	Why
Application shell	Next.js App Router	Suitable for marketing pages, authenticated dashboard, server-side data loading and client-heavy editor routes.
Editor route	Client-heavy island	Canvas libraries require browser APIs; isolate that complexity to <code>/board/[id]</code> .
Canvas engine	tldraw	Provides selection, transforms, zoom/pan, undo/redo, clipboard, custom shapes and document state.
UI state	Zustand	Use for panels, dialogs, editor preferences and transient app state - not as the source of truth for durable board data.
Schema validation	Zod	Validate API payloads and shared contracts.
Optimistic UI	Yes	User actions should feel immediate; server confirmation happens after the local change.
Autosave	Debounced/batched	Do not write on every pointer move; persist after short idle windows and important transitions.

Suggested route structure

```
/app
/(marketing)
/(auth)
/(dashboard)
  /dashboard
  /boards
  /templates
  /settings
  /board/[boardId]

/components
  /board
  /dashboard
  /templates
  /ui
```

```
/lib
/api
/auth
/boards
/storage
/schemas
```

Canvas object model

Object type	Examples	Persistence
Native/basic	Text, shape, image, drawing	Canvas document
Product-aware custom shape	Goal card, affirmation, checklist, quote card	Canvas document + optional domain reference
Asset reference	Image/video/file	Canvas stores asset ID/URL; bytes live in S3
Frame/section	Career, travel, health, finance	Canvas document
Link/embed	Website, video preview	Canvas metadata; remote content not embedded as raw bytes

6. Data model and persistence strategy

Persistence split

Use PostgreSQL for relational product metadata and permissions. Use S3 for uploaded media and optionally large immutable board snapshots. The editor document should reference assets rather than contain image bytes.

Data	Store	Why
Users, plans, profiles	Aurora PostgreSQL	Relational data and transactions.
Boards metadata	Aurora PostgreSQL	Ownership, title, visibility, timestamps, thumbnail, current version.
Membership/permissions	Aurora PostgreSQL	Many-to-many relationships and authorization queries.
Goals, comments, templates metadata	Aurora PostgreSQL	Queryable product features.

Canvas snapshot	JSONB in Postgres initially, or S3 if snapshots become large	Simple MVP persistence; move large/versioned snapshots to object storage when needed.
Images/videos/files	S3	Object storage is cheaper and more scalable than DB blobs.
Optimized variants	S3	Serve correct dimensions through CloudFront.
Jobs	SQS	Transient work queue with retry semantics.
Metrics/logs	CloudWatch / analytics platform	Operational and product telemetry should not live in app tables.

Recommended core relational schema

Table	Purpose	Important fields
users	Application profile mapped to Cognito identity	id, cognito_sub, display_name, plan, created_at
boards	Board-level metadata	id, owner_id, title, visibility, thumbnail_key, current_version, updated_at
board_members	Sharing and role-based access	board_id, user_id, role
board_versions	Optional version history	id, board_id, version, snapshot_key/jsonb, created_by, created_at
assets	Logical media records	id, owner_id, board_id, s3_key, mime_type, width, height, bytes, status
goals	Structured goal metadata	id, board_id, title, target_date, status
templates	Discoverable reusable board templates	id, owner_id, title, category, preview_key, snapshot_key
comments	Future collaboration comments	id, board_id, author_id, anchor, body, resolved_at
subscriptions	Billing state mirror	user_id, provider_customer_id, plan, status, period_end

Do not normalize every canvas object into SQL rows unless the product needs database queries over those objects. The editor engine should remain the authority for geometry and visual state. Relational tables should represent product concepts that the application needs to query independently.

7. Media upload, delivery, and image processing

Vision boards are primarily a media-delivery problem. A fast board depends more on asset architecture than on the database.

UPLOAD PATH

1. Browser asks API for upload permission
2. Lambda validates user/board/file metadata
3. Lambda returns an S3 presigned upload URL
4. Browser uploads directly to S3
5. S3 event -> SQS/Lambda image processor
6. Processor creates optimized variants + thumbnail
7. Asset row becomes READY
8. Board uses CloudFront URLs

DISPLAY PATH

Browser -> CloudFront -> cached optimized object -> S3 only on cache miss

Decision	Recommendation	Why
Originals	Keep original privately in S3	Allows future reprocessing/export without re-upload.
Board display	Serve WebP/AVIF/JPEG variants	Reduce bytes and decode cost.
Sizes	Generate 256 / 512 / 1024 / larger as needed	Use the smallest sufficient image for a board tile.
Uploads	Direct-to-S3 presigned URL	Avoid routing multi-MB files through Lambda/API Gateway.
Delivery	CloudFront custom domain	Cache close to users and centralize access rules.
Private assets	Signed CloudFront URLs/cookies or controlled origin access	Keep S3 bucket non-public where appropriate.
Image metadata	Store dimensions and byte size in assets table	Lets UI choose variants without probing files.
Deletion	Soft-delete metadata first, clean S3 asynchronously	Avoid user-facing latency and handle shared references safely.

Perceived-speed optimization

When a user drops an image onto the canvas, show a local object URL immediately. Upload in the background, then swap to the CDN URL after completion. The editing interaction should not wait for the network.

8. API and compute architecture

Workload	Compute choice	Reason
CRUD APIs	Lambda	Short, stateless requests.
Presigned upload URL	Lambda	Small authenticated operation.
Image processing	SQS-triggered Lambda	Asynchronous, retryable, does not block request.
Board export	SQS-triggered worker	Can be slow; return job status instead of holding HTTP open.
AI generation	Queued worker	External API latency and retry behavior should be isolated.
Scheduled cleanup	EventBridge + Lambda	Lifecycle/maintenance tasks.
Long-lived collaboration server	Do not use Lambda for a stateful socket server; use a managed collaboration service or Fargate if self-hosting	Persistent connections have different runtime requirements.

Database connectivity from Lambda

The API should not create uncontrolled PostgreSQL connections during traffic spikes. Use connection pooling appropriate to the chosen Aurora configuration. RDS Proxy can be introduced when connection churn or concurrency makes it useful. Keep functions in the same AWS region as the database and measure connection setup separately from query time.

API design rule	Reason
Return only fields needed for the screen	Serialization and transfer often cost more than the query.
Paginate lists	Avoid fetching hundreds of full records for initial render.
Use cursor pagination for large changing lists	More stable than deep OFFSET pagination.
Batch related reads where sensible	Avoid N+1 request patterns.
Run independent backend calls in parallel	Avoid sequential await chains.
Return job IDs for slow operations	Keep request latency predictable.
Set explicit timeouts/retries	Prevent hidden long-tail latency.

9. Performance design and the 750-post lesson

Key interpretation

A 3-4 second screen for roughly 750 posts does not prove that AWS is slow. It usually means the request is moving, serializing, rendering, or sequentially waiting on too much data. Measure each segment before changing infrastructure.

Latency decomposition

```
Total user-perceived latency ~=
network to API
+ API gateway / function startup
+ database connection acquisition
+ SQL execution
+ serialization
+ response transfer
+ browser parsing
+ React rendering / layout
+ media fetch/decode
```

Potential cause for 750-post slowness	What to measure	Typical fix
Fetching all 750 full records	Response row count and payload MB	Fetch 20-50 summaries; paginate or virtualize.
SELECT *	Columns and payload size	Return only list-view fields.
Missing index	EXPLAIN / query duration	Index filter/order keys.
N+1 queries	Query count per request	Join/batch/preload.
Lambda cold start	Init duration vs execution duration	Reduce package/startup work; provision only if justified.
Fresh DB connection each invocation	Connection acquisition time	Pool connections; consider RDS Proxy.
Sequential independent calls	Trace waterfall	Parallelize with Promise.all or backend batching.
Rendering all rows	Browser main-thread and React profiler	Virtualize list and render visible rows only.
Images loaded at full resolution	Network transfer and decode time	Thumbnails, responsive sizes, lazy loading, CDN.
Region distance	Network RTT	Place backend near primary users or use edge/CDN for cacheable content.

Performance targets for the vision-board app

Metric	Initial target	How to achieve it
Dashboard useful content	~1 second on a typical broadband connection	Small first payload, pagination, cacheable thumbnails.
Board shell usable	~0.5-1.5 seconds	Load board metadata/document before full-resolution assets.
Image placement	Immediate perceived response	Local preview + background upload.
Autosave	No visible editor stall	Debounce/batch persistence; send deltas or compact snapshot.
API p95 for simple reads	Aim for low hundreds of milliseconds	Indexes, small responses, pooled DB connectivity, tracing.
Heavy jobs	Async	Queue work; expose progress/status.

Board loading rule

Opening a board must not require downloading every original image. Fetch a compact board document first; load visible/nearby optimized media progressively through CloudFront.

10. Authentication, authorization, and security

Control	Decision	Why
Identity	Cognito user pools	Managed sign-up/sign-in and token issuance.
API authentication	JWT validation at API layer / Lambda	Every protected API call is tied to an authenticated principal.
Authorization	Check board membership/role server-side	Do not trust client-provided owner or role fields.
S3 access	Private bucket + presigned uploads + controlled CloudFront delivery	Avoid public writable storage.
Secrets	Secrets Manager / environment configuration	Do not ship DB credentials or API secrets to the browser.
Encryption	TLS in transit; AWS encryption at rest	Baseline protection for DB and S3.
Abuse controls	File type/size validation, rate limits, quotas	Media and AI endpoints are cost-sensitive.

Control	Decision	Why
Auditability	Log security-sensitive changes	Sharing, permissions, deletions, billing changes.

11. Async jobs and event-driven workflows

Job	Trigger	Worker output
Image optimization	S3 upload event -> SQS	Variants, dimensions, status READY.
Thumbnail generation	Board save / explicit request	Board preview image in S3.
PDF/image export	User request -> SQS	Export artifact + notification.
AI board generation	User prompt -> SQS	Structured board shapes + asset generation tasks.
Email	Application event -> queue	SES transactional email.
Cleanup	Scheduled EventBridge rule	Delete orphaned uploads/expired exports.
Analytics enrichment	Event stream/queue if needed later	Derived product metrics without slowing API.

Use idempotency keys for jobs that can be retried. A worker should be safe to execute more than once. Record job state such as QUEUED, RUNNING, SUCCEEDED and FAILED when the UI needs progress.

12. Realtime and collaboration strategy

Real-time multiplayer is valuable, but it should not dictate MVP infrastructure. Build the board model and permission model so collaboration can be added without rewriting ownership and sharing.

Phase	Capability	Recommended architecture
MVP	Single active editor; autosave	tldraw local state + API persistence.
Early sharing	View-only links / invited viewers	Postgres permissions + read API + CDN assets.
Comments/presence	Light realtime metadata	AWS AppSync/WebSocket-capable managed layer can be evaluated.

Phase	Capability	Recommended architecture
Full multiplayer canvas	Concurrent shape edits and conflict handling	Use a collaboration system designed for shared editor state; if self-hosting requires a persistent server, Fargate is preferable to managing EC2.

Decision boundary

Do not implement full multiplayer by bolting raw WebSockets onto Lambda and inventing a synchronization protocol. Use a purpose-built sync layer when the product requirement becomes real.

13. AI-ready architecture

AI should generate normal product data rather than create a parallel AI-only representation. This keeps generated content editable and compatible with templates, exports and collaboration.

```
User prompt
-> API creates generation job
-> SQS worker
    -> classify themes/goals
    -> generate copy / suggested structure
    -> optional image generation/search
    -> create ordinary board objects
    -> persist board version
-> UI receives job completion and opens editable board
```

AI feature	Architectural rule
Prompt-to-board	Output tldraw/custom-shape JSON that is valid without AI.
Image generation/editing	Treat resulting images exactly like user uploads: ingest -> process -> S3 -> CDN.
Goal decomposition	Persist structured goals separately only when the product needs tracking/querying.
AI coach/check-ins	Store generated recommendations as application records, not hidden model state.
Cost control	Queue, meter and rate-limit expensive operations.

14. Observability, reliability, and SLOs

Layer	Telemetry	Key questions
Browser	Sentry + web performance	Are users seeing JS errors, slow route loads, long tasks?
API Gateway/Lambda	CloudWatch logs/metrics/tracing	Which endpoint is slow? cold start or execution?
Aurora	Query metrics / slow-query analysis	Which SQL statements dominate latency?
S3/CloudFront	Request and cache metrics	Are assets cached? Are origin bytes excessive?
Queues	SQS depth/age/DLQ	Are background jobs keeping up?
Product	PostHog events/funnels	Do users create boards, upload assets, return, use templates?

Suggested service-level indicators

Indicator	Measure
API latency	p50/p95/p99 by endpoint
API errors	5xx and application-error rate
Board open	Time to interactive editor + time to visible first assets
Upload	Success rate and upload duration by size
Worker health	Queue age, retry rate, DLQ count
Database	Connection wait + query duration
CDN	Cache hit ratio and bytes from origin

15. Cost and scaling strategy

Cost driver	Control
Image storage	Lifecycle old exports/unused originals; compress variants.
Image egress	CloudFront caching; use appropriate resolutions.
Lambda	Keep requests short; do heavy tasks async; watch memory-duration tradeoff.

Cost driver	Control
Aurora	Choose capacity range based on measured usage; optimize SQL before scaling capacity.
AI	Per-user quotas, plan limits, queued work and usage metering.
Logs	Retention policies; structured logs without dumping huge request bodies.
Exports	Expire generated artifacts after a suitable period.

Scale by bottleneck, not by instinct. If the database is fast but browser rendering is slow, a larger database will not help. If CloudFront is serving cached images, adding API servers will not make images faster. Keep end-to-end traces and optimize the measured critical path.

16. MVP-to-production roadmap

Phase	Scope	Exit criterion
Phase 0 - foundation	Next.js, tldraw spike, Cognito, Aurora schema, S3 bucket, CloudFront, IaC	Prove editor integration and persistence.
Phase 1 - MVP	Create/edit boards, upload images, text/quotes/stickers, autosave, templates, dashboard, export image	Validate core use and retention.
Phase 2 - performance & polish	Image variants, direct uploads, pagination, caching, observability, SQS jobs, production error tracking	Make the product feel fast and operationally safe.
Phase 3 - monetization	Stripe, plan limits, premium templates, larger storage/AI quotas	Introduce paid value after usage patterns are known.
Phase 4 - collaboration	Sharing, roles, comments, presence; evaluate full multiplayer sync	Add only when user demand exists.
Phase 5 - AI	Prompt-to-board, AI image workflows, goal decomposition/coaching	Use existing board object model and async workers.

17. Anti-patterns and decisions to avoid

Avoid	Why
Store images inside PostgreSQL rows	Creates database bloat, backup/restore overhead and poor media-delivery behavior. Use S3.

Avoid	Why
Proxy all uploads through Lambda	Adds latency/cost and risks payload/time limits. Use presigned direct uploads.
Fetch entire datasets for list screens	Causes slow transfer and rendering. Paginate and project only needed fields.
Load originals on the canvas	Wastes bandwidth and decode time. Use responsive optimized variants.
Save on every pointer event	Creates excessive writes and editor jank. Debounce/batch.
Build a custom canvas engine first	Consumes months on editor primitives before validating product value.
Use DynamoDB just because it is AWS	The core domain is relational; use DynamoDB only for a workload that benefits from its access model.
Adopt multiple realtime systems immediately	Complicates source-of-truth and conflict semantics. Defer until collaboration scope is clear.
Treat Lambda memory as persistence	Function instances are ephemeral. Durable state must live in DB/object storage.
Scale infrastructure before profiling	Bigger compute cannot fix over-fetching, sequential requests or expensive browser rendering.

18. Architecture decision records (ADRs)

D1. Relational primary database

Decision	Selected approach	Why	Not chosen now
D1	Aurora PostgreSQL Serverless v2	The application has relational entities, sharing, roles, templates, goals and billing records that benefit from SQL and transactions.	DynamoDB as primary store: would require access-pattern-first modeling and more application-side joins/duplication before those benefits are needed.

Revisit trigger

Reconsider specialized DynamoDB tables only when a measured high-throughput access pattern clearly benefits from it.

D2. Media storage

Decision	Selected approach	Why	Not chosen now
D2	S3 + CloudFront	Separates large binary assets from application data; supports direct upload, CDN caching and lifecycle policies.	Database blobs or local disk: poor fit for durability, scaling and CDN delivery.

Revisit trigger

Revisit only for specialized derived media systems, not for basic uploads.

D3. Compute

Decision	Selected approach	Why	Not chosen now
D3	Lambda + API Gateway	Matches short stateless APIs and event workers without managing servers.	EC2: unnecessary operational burden for current workloads.

Revisit trigger

Use Fargate or another persistent runtime if a future component requires long-lived processes/connections.

D4. Canvas engine

Decision	Selected approach	Why	Not chosen now
D4	tldraw SDK	Provides mature editor primitives and lets the product focus on vision-board-specific shapes and workflows.	Custom canvas: high engineering cost and interaction complexity.

Revisit trigger

Reconsider only if product requirements fundamentally conflict with the SDK model or licensing/business constraints change.

D5. Board persistence

Decision	Selected approach	Why	Not chosen now
D5	Compact editor document + relational metadata	Keeps editor state cohesive while allowing product-level queries for boards, permissions, goals and assets.	One SQL row per visual primitive from day one: unnecessary impedance mismatch.

Revisit trigger

Extract specific object types into relational tables when they need independent querying/workflows.

D6. Realtime multiplayer

Decision	Selected approach	Why	Not chosen now
D6	Defer from MVP	Collaboration is valuable but not necessary to validate single-user creation, media, templates and retention.	Immediate custom WebSocket/sync infrastructure: large complexity before validated need.

Revisit trigger

Implement when shared editing becomes a prioritized user requirement with clear concurrency semantics.

D7. Performance strategy

Decision	Selected approach	Why	Not chosen now
D7	Small first payload + progressive media + async work	User-perceived speed depends on minimizing critical-path bytes and blocking work.	Load everything upfront: simple but degrades quickly as user content grows.

Revisit trigger

Keep this principle permanently; adjust cache, pagination and prefetch thresholds from telemetry.

19. Launch checklist

- Infrastructure is reproducible from CDK/Terraform and separated by environment.
- S3 bucket is not publicly writable; upload permissions are scoped and expire.
- CloudFront serves optimized board assets and cache headers are configured.
- Board list APIs are paginated and do not use SELECT *.
- Critical SQL queries have appropriate indexes and are measured with realistic data volumes.
- Lambda functions log endpoint/job identifiers and latency segments without leaking secrets.
- Failed background jobs go to a DLQ and can be replayed safely.
- Image processing generates multiple display sizes and stores dimensions/bytes.
- Board autosave is debounced and survives refresh/crash scenarios.
- Access-control tests cover owner/editor/viewer behavior and direct object access attempts.
- Frontend uses virtualization for any potentially large lists.
- Sentry/CloudWatch dashboards exist before public launch.
- Backups/retention policies exist for Aurora and critical S3 data.
- Cost alerts/quotas exist for AI, storage and unexpected API usage.

Appendix A. Suggested schema

```
users
  id uuid pk
  cognito_sub text unique
  display_name text
  plan text
  created_at timestampz

boards
  id uuid pk
  owner_id uuid fk users
  title text
  visibility text
  thumbnail_key text null
  canvas_snapshot jsonb -- MVP option
  current_version int
  created_at timestampz
  updated_at timestampz

board_members
  board_id uuid fk boards
  user_id uuid fk users
  role text -- owner/editor/viewer
  primary key (board_id, user_id)

assets
  id uuid pk
  owner_id uuid fk users
  board_id uuid null fk boards
  s3_key text
  mime_type text
  width int null
  height int null
  bytes bigint
  status text -- uploading/processing/ready/failed
  created_at timestampz

board_versions
  id uuid pk
  board_id uuid fk boards
  version int
  snapshot_json jsonb null
  snapshot_key text null
  created_by uuid fk users
  created_at timestampz
```

Indexing starting points

- boards(owner_id, updated_at desc) for dashboard lists.
- board_members(user_id, board_id) for shared-board discovery.
- assets(board_id, created_at) for board asset libraries.

- board_versions(board_id, version desc) for history.
- Use partial/unique indexes where business rules justify them; confirm with real EXPLAIN plans.

Appendix B. Suggested API surface

Method	Endpoint	Purpose
GET	/boards	Paginated dashboard board summaries.
POST	/boards	Create board from blank state or template.
GET	/boards/{id}	Board metadata + compact editor document reference/content.
PATCH	/boards/{id}	Update metadata.
PUT	/boards/{id}/snapshot	Persist debounced editor snapshot/version.
POST	/boards/{id}/assets/upload-url	Authorize direct S3 upload and create pending asset record.
POST	/assets/{id}/complete	Confirm upload; enqueue processing if required.
GET	/boards/{id}/assets	Paginated asset metadata.
POST	/boards/{id}/exports	Queue export and return job ID.
GET	/jobs/{id}	Return async job status/result.
POST	/boards/{id}/members	Invite/set role.
DELETE	/boards/{id}/members/{userId}	Remove access.

Appendix C. Performance investigation checklist

- Measure total API time and separate cold-start/init time from handler time.
- Measure database connection acquisition time separately from SQL execution.
- Capture exact response payload size in bytes, not only row count.
- Check query count per request for N+1 patterns.
- Run EXPLAIN (ANALYZE in safe environments) on slow SQL and verify index use.
- Inspect whether independent operations are awaited sequentially.
- Profile browser parse/render time and React commit time.
- Virtualize long lists; 750 items should not all need to be in the DOM initially.
- Check image dimensions, encoded file sizes and actual display sizes.

- Confirm CDN cache headers and cache-hit ratio.
- Check region RTT from primary user geography.
- Repeat tests with warm Lambda and cold Lambda separately.
- Load test with realistic concurrency rather than only a single developer request.
- Set a performance budget for initial JSON, initial image bytes and main-thread work.

Final recommendation

Start with the smallest architecture that preserves the right boundaries: Next.js + tldraw on the client; Cognito, API Gateway/Lambda, Aurora PostgreSQL, S3/CloudFront and SQS on AWS. Keep media direct-to-object-storage, keep initial payloads small, defer realtime multiplayer, and let measured bottlenecks determine future infrastructure.